

EFX₀ Allocations with at Most Three Relevant Goods: Existence and a Verified Polynomial-Time Algorithm

Evan Lin

University of Illinois Urbana-Champaign

Abstract. An allocation of indivisible goods is EFX₀ if no agent envies another agent’s bundle after the removal of any single good from it, even a good that the envious agent values at zero. We prove that every instance with nonnegative additive valuations in which each agent positively values at most three goods has a complete EFX₀ allocation, for any number of agents. The proof is an algorithm, K3ALG, that runs in polynomial time and returns such an allocation; its correctness and a bound of $400(n + m + 1)^4$ counted operations are machine-checked in Lean 4 on the program itself (for natural-number values, hence rational ones by scaling). K3ALG removes agents that value at most two remaining goods or have a dominant good, runs a serial dictatorship on the rest, places the unpicked goods into free slots and one large bundle, and, when the natural owner of that bundle fails, rotates picks once along a chain of agents. So when every agent values exactly three goods and none of them more than the other two together, some EFX₀ allocation has at most one bundle of more than two goods. Both theorems are machine-checked as well, over the reals, in core Lean with the standard axioms only. Bundles of at most two goods do not always suffice.

Keywords: Fair division · Indivisible goods · EFX · Serial dictatorship · Formal verification · Lean

1 Introduction

Envy-freeness up to any good is a central relaxation of envy-freeness for indivisible goods [4]. Let agents $N = [n]$ share goods $M = [m]$, with additive valuations $v_i(S) = \sum_{g \in S} v_i(g)$, $v_i(g) \geq 0$. An allocation $X = (X_1, \dots, X_n)$, a partition of M , is EFX₀ if

$$v_i(X_i) \geq v_i(X_j \setminus \{h\}) \quad \text{for all } i \neq j \text{ and all } h \in X_j,$$

and EFX if this is required only for goods h with $v_i(h) > 0$; the stronger notion is usually called EFX₀ [4]. The difference matters when agents value few goods: if X_j contains a good worthless to i , then EFX₀ requires

$v_i(X_i) \geq v_i(X_j)$, so goods that nobody values cannot be handed to an arbitrary agent.

Complete EFX allocations are known to exist for additive valuations only in special cases, among them three agents [5], $m \leq n + 3$ [12], four agents with at most nine goods [3], when every good is valued by at most two agents [6,1,7,13], and on hypergraphs of girth at least four [8]; these are EFX₀ results. For monotone valuations EFX can fail [2,11]. Call an instance *k-relevant* if every agent positively values at most k goods. For $k \leq 2$, serial dictatorship gives an EFX₀ allocation [9]. For $k \leq 4$, Viswanathan and Mehta [14] announce ordinary EFX, with a proof sketch; their algorithm gives the goods nobody values to an arbitrary agent, which can break EFX₀. We ask whether every 3-relevant instance has a complete EFX₀ allocation. A counterexample would refute the additive EFX conjecture, since perturbing all values by a small $\varepsilon > 0$ (compare [5, Lemma 1]) turns it into a positive instance without EFX allocations; but the perturbation destroys the relevance structure, and we know of no result that implies a positive answer (see the literature search in [10]).

Theorem 1 (existence). *Every instance with $n \geq 1$ agents and non-negative real additive valuations in which every agent positively values at most three goods has a complete EFX₀ allocation.*

Call an agent *balanced* if it values exactly three goods and none of them more than the other two together.

Theorem 2 (one large bundle). *In every instance with $n \geq 1$ agents and nonnegative real additive valuations in which every agent is balanced, some EFX₀ allocation has at most one bundle with more than two goods.*

Theorem 3 (algorithm). *On every instance with $n \geq 1$ agents and natural-number values in which every agent positively values at most three goods, algorithm K3ALG (Section 3) returns an EFX₀ allocation. On every instance with $n \geq 1$ agents it performs at most $400(n + m + 1)^4$ counted operations.*

Scaling each agent's values preserves EFX₀, so Theorem 3 covers rational values.

Contributions. (i) Existence over the reals (Theorem 1), with the structure of Theorem 2. (ii) A polynomial-time algorithm, K3ALG, with correctness and operation bound machine-checked on the program itself

(Theorem 3); its correctness proof (Section 4) also gives (i). (iii) A formalization in core Lean 4 with the standard axioms only (Section 6). (iv) Limits: bundles of at most two goods do not always suffice (smallest counterexample: three agents), nor does one bundle of three goods (Proposition 3).

Status of the claims. The work was done in a public repository [10] whose ledger gives every claim a status: PROVED (complete written proof), CERTIFIED (exhaustive computation over a provably complete set, with independently checked certificates), EVIDENCE, CONJECTURE or REFUTED. All positive results below are PROVED; two independent reviews of the proof of Theorems 1 and 2 found no error.

2 Preliminaries

Let $R_i = \{g : v_i(g) > 0\}$. Since $\max_{h \in X_j} v_i(X_j \setminus \{h\}) = v_i(X_j) - \min_{h \in X_j} v_i(h)$, a bundle of one good is never strongly envied, and i must not envy at all a bundle that contains a good outside R_i . If $|R_i| = 3$ we write $R_i = \{a_i, b_i, c_i\}$ with $v_i(a_i) \geq v_i(b_i) \geq v_i(c_i) > 0$, fix this order (ties broken by index) as $a_i \succ_i b_i \succ_i c_i$, and call i *balanced* if $v_i(a_i) \leq v_i(b_i) + v_i(c_i)$, and *strictly balanced* if the inequality is strict.

Lemma 1 (peeling). *Let $i \in N$ and $P \subseteq M$ satisfy (i) $v_i(P) \geq v_i(M \setminus P)$ and (ii) $|P| \leq 1$ or P is worthless to every other agent. If the instance without i and P has an EFX₀ allocation Y , then Y together with $X_i = P$ is EFX₀.*

Rule R1 applies Lemma 1 with $P = \{p\}$ for a favourite remaining good p of i when $v_i(p) \geq v_i(M \setminus \{p\})$, and with $P = \emptyset$ when $v_i(p) = 0$. If R1 fails for every agent, then $2v_i(g) \leq 2v_i(p) < v_i(M)$ for all g , so in a 3-relevant instance every agent values exactly three goods and is strictly balanced.

Lemma 2 (real values reduce to naturals). *If v is nonnegative real additive with $|R_i| \leq 3$ for all i , there are natural-number additive valuations w with the same relevant goods such that $v_i(S) \leq v_i(T) \iff w_i(S) \leq w_i(T)$ for every agent i and all $S, T \subseteq M$.*

Such comparisons depend only on the ties among a_i, b_i, c_i and the sign of $v_i(a_i) - v_i(b_i) - v_i(c_i)$ (Appendix A).

3 Algorithm K3ALG

K3ALG has two stages: *Stage R* peels agents by rule R1, and *Stage L* runs construction LB^+ on the remaining agents A and goods M ; each of these agents values exactly three goods and is strictly balanced. Stage L first runs a serial dictatorship with *R1 priority*: the next agent has at most two of its goods left if such an agent exists (an *R1 step*), and otherwise it finds all three available and takes its top (an *insertion step*). Each agent i takes its $\succ_i$ -favourite remaining good as its *pick* Y_i , or no pick ($Y_i = \perp$). A *block* is an insertion step with the R1 steps that follow it, and its inserted agent is its *leader*. The upgrades then give some agents u with $Y_u = b_u$ also c_u ; these agents form U . The *junk* J is the set of goods that are neither picks nor such goods c_u . For $i \notin U$ the *needs* are $N_i = \{g \in R_i : g \succ_i Y_i\}$ ($N_i = R_i$ if $Y_i = \perp$), and $NA = \bigcup_{i \notin U} N_i$ is the set of goods *needed alone*. An agent $i \notin U$ is *frozen* if $Y_i \in NA$, and a *terminal* otherwise. The *slots* are $\text{cap}(i) = 0$ for upgraded and frozen agents, and 1 or 2 for a terminal with or without a pick; $S = \sum_i \text{cap}(i)$. The *base* of i is $\{Y_i\}$ (or $\emptyset$) for $i \notin U$ and $\{b_i, c_i\}$ for $i \in U$. A *completion* with *owner* o (a terminal or upgraded agent) gives every agent its base, at most $\text{cap}(i)$ junk goods to every terminal $i \neq o$, and the rest of the junk to o ; it satisfies the *owner constraint* (OC) if no agent $x \notin U$, $x \neq o$, with $Y_x = a_x$ has $\{b_x, c_x\} \subseteq X_o$. The agents *exposed* for o are $E_o = \{x \notin U, x \neq o : Y_x = a_x, \{b_x, c_x\} \subseteq J \cup \text{base}(o)\}$, and $\pi_x = \{b_x, c_x\} \cap J$. A *need chain* from a frozen agent x is a sequence $x = x_0, \dots, x_t$ of distinct agents in which each $x_{s+1} \notin U$ needs Y_{x_s} , $x_0, \dots, x_{t-1}$ are frozen, and x_t is a terminal.

Algorithm K3ALG (as formalized in EFX.K3.algoC). Input: $n \geq 1$ agents, m goods, values $v_i(g) \in \mathbb{N}$; A and M start as all agents and all goods. Lists are in index order, and “first” means smallest index.

- R1. If $|A| \leq 1$, give M to the agent of A and stop. If $M = \emptyset$, stop.
- R2. Find the first $i \in A$ such that, for the first good $p \in M$ of largest $v_i(p)$, $v_i(p) = 0$ or $v_i(M \setminus \{p\}) \leq v_i(p)$; if none, go to L1.
- R3. Remove i from A ; if $v_i(p) > 0$, let $X_i = \{p\}$, $M \leftarrow M \setminus \{p\}$. Go to R1.
- L1. For $i \in A$, sort i 's relevant goods in M as a_i, b_i, c_i (ties by index).
- L2. *Phase 1*. Repeatedly, the first unprocessed agent with at most two of its goods unpicked (else the first unprocessed agent) picks the first of a_i, b_i, c_i still unpicked, if any.
- L3. *Upgrades*. $U \leftarrow \emptyset$. While some $k \notin U$ has $Y_k = b_k$, $c_k \in J$ and $b_k \notin NA$, add the first such k to U .

- L4. If $|J| \leq S$, return COMPLETE(no owner, $\emptyset$).
- L5. Let r be the last-processed agent not in U , and $H = \text{HITSET}(E_r)$. If $|H| \leq S - \text{cap}(r)$, return COMPLETE(r, H).
- L6. *Rotation*. Let k be the first agent of E_r in r 's block. Scanning the agents processed after k in order, append j to the chain $k, \dots$ if the chain's last agent is frozen, $j \notin U$, and j needs its pick. Each chain agent takes its predecessor's pick; k takes b_k, c_k and joins U .
- L7. Recompute J , cap and S . If $|J| \leq S$, return COMPLETE(no owner, $\emptyset$); otherwise return COMPLETE($k, \text{HITSET}(E_k)$).

HITSET(E): for each $z \in E$, b_z if $b_z \in J$ and c_z otherwise; but if two agents $x \neq y$ of E (the first such pair) have a common good in $\pi_x \cap \pi_y$, that good replaces the choices for x and y . COMPLETE(o, H): picks go to their pickers, c_u to each $u \in U$; the junk, H first and then the rest of J , fills the slots of the agents other than o in index order; o gets the remaining junk.

4 Correctness

Stage R is correct by Lemma 1. For Stage L we prove more, for every Phase 1 order with R1 priority, upgrade order and need chain (construction LB⁺). Phase 1 has four immediate invariants (Appendix B): (I1) if $g \succ_i Y_i$ (or $Y_i = \perp$), then g was picked before i 's turn, and (B) by an agent of i 's block; (I2) if $g \in R_i$ is never picked, then $Y_i \succ_i g$; (I3) an agent that finds all its goods available is a leader. A pair (Y, U) as above is *valid* if (V1) $J \cap NA = \emptyset$ and (V2) $b_u, c_u \notin NA$ for $u \in U$. After L3 it is valid, by (I2) and because NA only shrinks (Lemma 5).

Theorem 4 (soundness). *Let (Y, U) be valid. Every completion that satisfies (OC) is EFX₀ for every balanced additive valuation consistent with the rankings, and only the owner's bundle can have more than two goods.*

Proof (idea). A bundle of two or more goods contains no good needed alone, by (V1) and (V2), so it can threaten agent i only through b_i and c_i together, while i holds only a_i ; with two goods that is harmless, and with more it is the owner's bundle, which (OC) excludes (Appendix B). $\square$

Lemma 3 (owner criterion). *Let (Y, U) be valid with $|J| > S$. If no $x \in E_o$ has $\{b_x, c_x\} \subseteq \text{base}(o)$, and some $H \subseteq J$ with $|H| \leq S - \text{cap}(o)$ meets $\{b_x, c_x\}$ for every $x \in E_o$, then a completion with owner o satisfies (OC).*

(Fill the other slots with H first.) Then o is a *valid owner*. If $|J| \leq S$, the completion without owner has all bundles of at most two goods.

Theorem 5 (the owner r). *Let $|J| > S$ after Phase 1 and the upgrades. Then r is a valid owner unless the bad case holds: an agent k^* of r 's block is exposed for r and frozen, every need chain from k^* ends at r , and the sets π_x ($x \in E_r$) are pairwise disjoint.*

Proof (idea). An exposed agent found all its goods available, so it leads its block (I3). For each exposed x outside r 's block, x or the end of a need chain from x is a terminal in x 's block (B); these terminals are distinct and differ from r , so $S - \text{cap}(r) \geq |E_r| - 1$, or $\geq |E_r|$ without k^* . One junk good per π_x , with a shared good where two meet, then fits unless the bad case holds (Appendix B). $\square$

Theorem 6 (rotation). *In the bad case, rotate along any need chain $k^* = x_0, \dots, x_t = r$: each x_s ($s \geq 1$) takes $Y_{x_{s-1}}$, and k^* takes b_{k^*}, c_{k^*} and joins U . The result is valid, and if its junk exceeds its slots, then k^* is a valid owner.*

Proof (idea). Chain agents get goods they needed, so NA shrinks, and $b_{k^*}, c_{k^*} \in J \cup \{Y_r\}$ are free. Agents exposed for k^* afterwards were exposed for r , disjointness keeps their pairs out of $\{b_{k^*}, c_{k^*}\}$, and the terminals found for them lie outside r 's block and keep their slots (Appendix B, with Example 1). $\square$

LB^+ is Stage L with any Phase 1 order with R1 priority and upgrade order, owner r if valid, and otherwise a rotation along any need chain from k^* to r and owner k^* .

Theorem 7 (LB^+ never fails). *If every agent values exactly three goods and is balanced, then for all these choices LB^+ returns an allocation that is EFX_0 for every balanced additive valuation consistent with the rankings, with at most one bundle of more than two goods.*

Proof. The pair (Y, U) is valid, and the completion satisfies (OC), by Lemma 3 and Theorems 5 and 6. Apply Theorem 4. $\square$

Proof (of Theorems 1–3 (correctness)). L2 has R1 priority, L5 tests r exactly (Proposition 1), and L6 builds a need chain from k^* to r in the bad case, so Stage L is a run of LB^+ (Theorem 7, hence Theorem 2). With Lemma 1 and induction on n , K3ALG is correct (Theorem 3); Lemma 2 gives Theorem 1. $\square$

5 Running Time

A minimum set of junk goods meeting all sets π_x (of at most two goods each) is a vertex cover in general, but the owner test for r needs none.

Proposition 1 (the owner test). *With $|J| > S$ after Phase 1 and the upgrades, r is a valid owner if and only if $|E_r| \leq S - \text{cap}(r)$ or two of the sets π_x ($x \in E_r$) meet; $\text{HITSET}(E_r)$ is then a witness.*

Proof. The proof of Theorem 5 gives $S - \text{cap}(r) \geq |E_r| - 1$, and nonempty sets π_x have a hitting set of fewer than $|E_r|$ goods if and only if two of them meet. $\square$

In Lean, K3ALG is a program `EFX.K3.algoC` in a *cost monad* (a value paired with an operation count), and `EFX.K3.algo` is defined as its value, so the bound of Theorem 3 is about the program that computes the allocation. One unit is a list step, a comparison of agents or goods, a read of an input value or of a table, or a comparison or addition of natural numbers of any size (Appendix C). The dominant term is the upgrade loop L3. The finer bound $O(n^4 + n^2m)$ is written but not formalized, as is $O((n + m) \log n)$ after reading the input for a Python implementation, which agrees with the Lean program on all instances tested and takes about 1.6s with 10^5 agents (evidence). K3ALG compares only subset sums of one agent’s values, so by Lemma 2 it runs verbatim on real values (a written remark, not in the ledger).

6 The Lean Verification

The formalization uses core Lean 4 (v4.34.0) and no packages; no file contains `sorry`, every declaration uses only the three standard axioms (checked by a script), and Lean’s replay checker re-checks the library. The trusted base is six short definitions copied from [9] (Appendix D): values $v : \text{Fin } n \rightarrow \text{Fin } m \rightarrow \mathbb{N}$, an allocation as a map from goods to agents, and EFX₀ with the removed good ranging over *every* good of the other bundle. Theorem 3 is (in namespace `EFX.K3`):

```
def algo (I : Inst) (hn : 0 < I.n) : I.Alloc :=
  (algoC I hn).val
theorem algo_efx0 (I : Inst) (hn : 0 < I.n)
  (h : ∀ i, numRelevant I i ≤ 3) : I.EFX0 (algo I hn)
theorem algoC_cost (I : Inst) (hn : 0 < I.n) :
  (algoC I hn).cost ≤ 400 * (I.n + I.m + 1) ^ 4
```

`EFX.K3.algo_eq_spec` proves that the counted program computes exactly the specification `EFX.K3.algoSpec`, the composition of the peeling loop, the processing order and LB^+ . So the program whose cost is bounded is the one proved correct. Theorems 1 and 2 are `EFX.target` and `EFX.LB.corollaryD` over $\mathbb{N}$, and `EFX.target_ordered` and `EFX.corollaryD_ordered` over any linearly ordered cancellative commutative monoid via Lemma 2 (`EFX.112`); that $\mathbb{R}_{\geq 0}$ is one is the textbook fact, as core Lean has no reals. Every result of Section 4 is formalized (Appendix D). An independent restatement of Theorems 1 and 2 follows from them, and a second kernel re-checked their natural-number development. Not formalized: the size of the large bundle, the finer running-time bounds and the real-value remark.

7 Certification and Evidence

Independently of the proof, a SAT-free checker verified from the raw EFX_0 definition that stored allocations cover all 6^n ranking profiles (Lemma 6) of every connected core (Appendix A), as enumerated by nauty’s `genbg` and cross-checked (Appendix E). This certifies EFX_0 for all 3-relevant instances with $n \leq 7$, and Theorem 2 for all cores with $n \leq 6$ and all connected cores with $n = 7$, or $n = 8$ and $m \geq 13$. As evidence, K3ALG returned an EFX_0 allocation on all 152,071,632 ranking profiles of all cores with $n \leq 6$.

8 Conclusion and Open Problems

Every 3-relevant instance has a complete EFX_0 allocation, and a verified polynomial-time algorithm finds one. Four relevant goods per agent is open (the repository has work in progress on it, about which we claim nothing), as are the minimum size of the large bundle and the complexity of the owner test for arbitrary owners.

Acknowledgments. The proofs, code and Lean formalization were developed with the assistance of AI coding agents (Claude Code); every claim here is machine-checked, computer-certified or refereed, as stated.

References

1. Afshinmehr, M., Ashuri, A., Mahmoudkhan, P., Mehlhorn, K., Shahrezaei, A.M.: EFX allocations exist on multi-graphs. arXiv:2606.18665 (2026)

2. Akrami, H., Mayorov, A., Mehlhorn, K., Srinivas, S., Weidenbach, C.: A counterexample to EFX $n \geq 3$ agents, $m \geq n + 5$ items, submodular valuations via SAT-solving. arXiv:2604.18216 (2026)
3. Alkassar, E., Fouz, M., Mehlhorn, K.: Complete EFX allocations exist for four additive agents and up to nine goods. arXiv:2608.08590 (2026)
4. Amanatidis, G., Birmpas, G., Filos-Ratsikas, A., Voudouris, A.A.: Fair division of indivisible goods: A survey. arXiv:2202.07551 (2022)
5. Chaudhury, B.R., Garg, J., Mehlhorn, K.: EFX exists for three agents. arXiv:2002.05119 (2020)
6. Christodoulou, G., Fiat, A., Koutsoupias, E., Sgouritsa, A.: Fair allocation in graphs. In: Proceedings of the 24th ACM Conference on Economics and Computation (EC 2023). pp. 473–488 (2023). <https://doi.org/10.1145/3580507.3597764>
7. Kaviani, A., Seddighin, M., Shahrezaei, A.: Almost envy-free allocation of indivisible goods: A tale of two valuations. arXiv:2407.05139 (2024)
8. Lianes, T., Sgouritsa, A., Sotiriou, M.M.: EFX allocation in (multi)hypergraphs. arXiv:2608.03171 (2026)
9. Lin, E.: mrd-efx. GitHub repository (Lean formalization for at most two relevant goods per agent), <https://github.com/evanlin23/mrd-efx>
10. Lin, E.: efx0-three-goods: EFX₀ with at most three relevant goods per agent (proofs, certificates, Lean formalization). GitHub repository, <https://github.com/evanlin23/efx0-three-goods> (2026)
11. Mackenzie, S., Suzuki, M.: Counterexamples to EFX for submodular and subadditive valuations. arXiv:2605.06451 (2026)
12. Mahara, R.: Extension of additive valuations to general valuations on the existence of EFX. arXiv:2107.09901 (2021)
13. Sgouritsa, A., Sotiriou, M.M.: On the existence of EFX allocations in multigraphs. arXiv:2502.09777 (2025)
14. Viswanathan, M., Mehta, R.: On the existence of EFX under picky or non-differentiative agents. In: Proceedings of the International Conference on Autonomous Agents and Multiagent Systems (AAMAS 2024). pp. 2534–2536 (2024), extended abstract

A Proofs for Section 2

Full proofs of all reduction lemmas are in `proofs/lemmas.md` of [10]; we reproduce those used here.

Proof (of Lemma 1). Every other bundle lies in $M \setminus P$, so i values it at most $v_i(M \setminus P) \leq v_i(P) = v_i(X_i)$. Towards P , every other agent sees a single good or a bundle worthless to it. The remaining constraints are those of Y . $\square$

Lemma 4 (junk). *Let $J \neq \emptyset$ be the goods valued by no agent. If the instance without J has an EFX_0 allocation, so does the instance.*

Proof. Let Y be EFX_0 for $M \setminus J$. While the envy graph ($i \rightarrow j$ iff $v_i(Y_j) > v_i(Y_i)$) has a cycle, give each agent on it the bundle of its successor. The bundles are the same and no agent loses value; an agent that now sees its old bundle satisfies $v_i(Y_i \setminus \{h\}) \leq v_i(Y_i)$, so EFX_0 is preserved. The sum of values strictly increases, so this stops, with an acyclic envy graph. Let s be a source: $v_i(Y_s) \leq v_i(Y_i)$ for all i . Put $X_s = Y_s \cup J$. For $i \neq s$, $v_i(X_s \setminus \{h\}) \leq v_i(X_s) = v_i(Y_s) \leq v_i(Y_i)$; all other constraints are unchanged. $\square$

Cores. Agent i is *top-heavy* if $v_i(a_i) \geq v_i(b_i) + v_i(c_i)$. Rule R2 gives i the set P of its relevant goods that no other agent values, if $|P| \geq 2$ and (i) of Lemma 1 holds. A *core* is an instance with at least two agents, each valuing exactly three goods and strictly balanced, each with at most one *private* good (one no other agent values), and no good valued by nobody.

Proposition 2 (core reduction). *If every core with at most N_0 agents has an EFX_0 allocation, so does every 3-relevant instance with between 1 and N_0 agents.*

Proof (of Proposition 2). Induction on $n + m$ over 3-relevant instances with $1 \leq n \leq N_0$. One agent takes everything. Otherwise: (1) if some good is valued by nobody, use Lemma 4; (2) else if some agent has at most two relevant goods or is top-heavy, use rule R1 and Lemma 1; (3) else if some agent i has at least two private goods, let P be all of them: if $|P| = 3$, (i) is trivial, and if $P = \{x, y\}$ and $R_i \setminus P = \{z\}$, then $v_i(x) + v_i(y) \geq v_i(b_i) + v_i(c_i) > v_i(a_i) \geq v_i(z)$, so Lemma 1 applies (rule R2); (4) else the instance is a core, covered by hypothesis. Each case passes to a smaller 3-relevant instance. $\square$

In Lean this is `EFX.core_reduction` (with `EFX.peel`, `EFX.peelEmpty`, `EFX.peelR2` and `EFX.Inst.exists_efx0_of_junk`).

Proof (of Lemma 2). Fix i ; w_i is built from v_i alone. Goods outside R_i contribute 0 to both sides under both valuations, and removing goods common to S and T changes neither comparison, so it suffices that w_i preserves $v_i(X) \leq v_i(Y)$ for disjoint $X, Y \subseteq R_i$. Name the relevant goods with values $a \geq b \geq c > 0$. Comparisons with an empty side are decided by positivity; two single goods by the ties among a, b, c ; a single good against the other two by the sign of $a - (b + c)$, because $b < a + c$ and $c < a + b$ always hold. Two disjoint pairs do not fit in three goods. Choose w_i on R_i (in the same order) by the pattern: $(4, 3, 2)$ if $a > b > c$ and $a < b + c$; $(3, 2, 1)$ if $a > b > c$ and $a = b + c$; $(5, 2, 1)$ if $a > b > c$ and $a > b + c$; $(2, 2, 1)$ if $a = b > c$; $(3, 2, 2)$, $(2, 1, 1)$, $(3, 1, 1)$ if $a > b = c$ and $a < 2b$, $a = 2b$, $a > 2b$; $(1, 1, 1)$ if $a = b = c$; $(2, 1)$ or $(1, 1)$ for two relevant goods; (1) for one; and $w_i(g) = 0$ for $g \notin R_i$. Each triple has the same ties and the same sign of $a - (b + c)$ as its pattern. $\square$

Lean: `EFX.112`, which uses all 31 permutations of these representatives (checked by `decide`) instead of sorting, and `EFX.efx0_iff_of_agree` for the transfer of EFX_0 .

B Proofs for Section 4

These are the proofs of `proofs/lb_last_step.md` of [10], Sections 1–6. A pair $P = (Y, U)$ as in Section 3 is a *pre-allocation*; F and T denote its frozen agents and its terminals, and $\omega = |J| - S$. After the rotation, primes denote the new state: $P' = (Y', U')$, J' , NA' , cap' , S' , ω' , E'_k .

Phase 1. (I1): i took its favourite remaining good, and goods leave G only by being picked. (I2): g was still in G at i 's turn. (I3): R1 steps process only agents with at most two goods left. (B): each block starts with an insertion step, when every unprocessed agent i has $R_i \subseteq G$; by (I1) g was picked before j 's turn, and not in an earlier block, since j was unprocessed when its own block started.

Proof (of Theorem 4). Only X_o can exceed two goods. Fix i and $j \neq i$ with $|X_j| \geq 2$. Then j is not frozen, so $X_j \cap NA = \emptyset$ by (V1) and (V2). If $i \in U$, then X_j meets R_i in at most a_i , and $v_i(X_j \setminus \{h\}) \leq v_i(a_i) \leq v_i(b_i) + v_i(c_i) \leq v_i(X_i)$. If $Y_i = \perp$, then $R_i = N_i \subseteq NA$ and X_j is worthless to i . Otherwise every good of $R_i \cap X_j$ is outside $N_i \cup \{Y_i\}$, hence below

Y_i . If there is at most one, $v_i(X_j \setminus \{h\}) \leq v_i(Y_i) \leq v_i(X_i)$. If there are two, then $Y_i = a_i$ and $R_i \cap X_j = \{b_i, c_i\}$; for $|X_j| = 2$ removing one leaves a good worth at most $v_i(a_i)$, and $|X_j| \geq 3$ means $j = o$, excluded by (OC). $\square$

Proof (of Lemma 3). Since $|J| = S + \omega > S - \text{cap}(o)$, H extends to $C \subseteq J$ with $|C| = S - \text{cap}(o)$, which exactly fills the slots of the terminals other than o ; let $X_o = \text{base}(o) \cup (J \setminus C)$. If $x \notin U$, $x \neq o$, $Y_x = a_x$ had $\{b_x, c_x\} \subseteq X_o$, then $\{b_x, c_x\} \subseteq J \cup \text{base}(o)$, so $x \in E_o$, and $H \subseteq C$ meets $\{b_x, c_x\}$ outside X_o , a contradiction. $\square$

Lemma 5 (upgrades). *When the upgrade loop stops, (Y, U) is a valid pre-allocation, and (UT) no $k \notin U$ has $Y_k = b_k$, $c_k \in J$ and $b_k \notin NA$.*

Proof. (UT) is the stopping condition. (V1): by (I2) a junk good lies below the pick of every agent that values it, so it is in no N_i . (V2): c_u was junk; $b_u \notin NA$ held when u was upgraded, and NA only shrinks as U grows. $\square$

Proof (of Theorem 5). r exists: the first processed agent is inserted and holds its top, so it is not upgraded. (A1) $r \in T$: if $Y_r = \perp$, r is not frozen by definition; if $Y_r \in N_j$ for some $j \notin U$, then by (I1) j was processed after r , contradicting the choice of r . (A2) Every agent processed after r is in U . (A3) Let $x \in E_r$. Then x was processed before r (A2). At x 's turn, $a_x = Y_x$, every junk good, and Y_r were available, and $\{b_x, c_x\} \subseteq J \cup \{Y_r\}$, so $R_x \subseteq G$ and x is a leader by (I3). Hence each block contains at most one exposed agent; let B^* be r 's block and k^* its exposed agent, if any. (A4) For a frozen x , some $j \notin U$ has $Y_x \in N_j$, and by (B) j is processed after x in x 's block. Iterating while the current agent is frozen gives a need chain of distinct agents of x 's block ending at a terminal. Let $\tau(x) = x$ if $x \in T$, and otherwise the end of some need chain from x ; it has $\text{cap} \geq 1$. For $x \in E_r$, $\pi_x = \{b_x, c_x\} \cap J \neq \emptyset$, since at most one of b_x, c_x is Y_r ; for the same reason no exposed pair lies in $\text{base}(r)$, the first condition of Lemma 3.

The agents $\tau(x)$, $x \in E_r \setminus B^*$, lie in distinct blocks other than B^* , so they are distinct terminals other than r , and $S - \text{cap}(r) \geq |E_r \setminus B^*|$. Let H contain one good of each π_x , a common one when two meet. If $E_r \cap B^* = \emptyset$, then $|H| \leq |E_r| \leq S - \text{cap}(r)$. Otherwise $E_r \cap B^* = \{k^*\}$, $k^* \neq r$ and $|E_r \setminus B^*| = |E_r| - 1$. If $k^* \in T$, or some need chain from k^* ends at a terminal other than r , then $\tau(k^*)$ can be chosen in $B^* \setminus \{r\}$, one more terminal, so $S - \text{cap}(r) \geq |E_r| \geq |H|$. If two sets π_x meet, then $|H| \leq |E_r| - 1 \leq S - \text{cap}(r)$. If none of these holds, the bad case holds. $\square$

Proof (of Theorem 6). Let $k = k^*$, $W = J \cup \{Y_r\}$; the new junk is $J' = W \setminus \{b_k, c_k\}$, and every pick other than Y_r is still a pick in P' . (a) P' is a pre-allocation: $Y'_{x_s} \in R_{x_s}$ since x_s needed it, and $b_k, c_k \in W$ are neither picks of P' nor goods c_u . (b) $NA' \subseteq NA$: $Y'_{x_s} \succ Y_{x_s}$ (or $Y_{x_s} = \perp$), so $N'_{x_s} \subseteq N_{x_s}$; k had $N_k = \emptyset$ and is now in U' . (c) Validity: $J' \subseteq W$, $J \cap NA = \emptyset$ and $Y_r \notin NA$ (A1), so $J' \cap NA' = \emptyset$; for $u \in U$, $b_u, c_u \notin NA \supseteq NA'$; $b_k, c_k \in W$ are not in NA . (d) A terminal $x \notin B^*$ keeps its pick and is a terminal of P' with the same slots. (e) $E'_k \subseteq E_r \setminus \{k\}$, where E'_k is taken in P' , with $J' \cup \{b_k, c_k\} \subseteq W$. An agent off the chain in E'_k is in E_r . An interior chain agent x_s ($1 \leq s < t$) is not: $a_{x_s} = Y'_{x_s} \succ Y_{x_s}$, so $Y_{x_s} \in \{b_{x_s}, c_{x_s}\}$, and $Y_{x_s} \notin W$. Nor is r : $Y_r \neq a_r$. If $Y_r = \perp$ or $Y_r = c_r$, then b_r was picked by another agent (I1), so $b_r \notin W$. If $Y_r = b_r$, then (UT) applies to r ($b_r \notin NA$ by (A1)), so $c_r \notin J$, and $c_r \neq Y_r$, so $c_r \notin W$. (f) If some $x \in E'_k$ had $\{b_x, c_x\} = \{b_k, c_k\}$, then $x \in E_r \setminus \{k\}$ and $\pi_x = \pi_k \neq \emptyset$, against disjointness. (g) By (f), every $x \in E'_k$ has a good of $\{b_x, c_x\}$ in J' ; let H' contain one each. By (e), $|H'| \leq |E_r| - 1 = |E_r \setminus B^*|$. The terminals $\tau(x)$, $x \in E_r \setminus B^*$, are distinct, outside B^* , and keep $\text{cap}' \geq 1$ by (d). So $|H'| \leq S' = S' - \text{cap}'(k)$, and Lemma 3 applies to P' and $o = k$. $\square$

Example 1 (a rotation). Let agents 0, 1, 2 rank $g_0 \succ g_2 \succ g_3, g_2 \succ g_1 \succ g_4, g_0 \succ g_2 \succ g_1$, processed in the order 1, 0, 2 (a Phase 1 order with R1 priority, though not the one of L2). Agent 1 is inserted and picks g_2 ; agents 0 and 2 then pick g_0 and g_1 in R1 steps. Now $J = \{g_3, g_4\}$, $NA = \{g_0, g_2\}$, agents 0, 1 are frozen, and $r = 2$ has one slot. Giving r both junk goods violates (OC) for agent 1 ($b_1 = g_1, c_1 = g_4$). The rotation along the chain $1 \rightarrow 2$ gives $\{g_0\}, \{g_1, g_4\}, \{g_2, g_3\}$, which is EFX₀.

Size of the large bundle (written, not formalized). In a valid pre-allocation each good of NA is the pick of exactly one frozen agent, so $|F| = |NA|$; counting goods gives $\omega = m - 2n + |NA|$, and the owner's bundle has $\omega + 2$ goods. The rotation does not increase NA .

C Running Time: Cost Table and Evidence

The program is the listing of Section 3, formalized as `EFX.K3.algoC`; see `proofs/k3_algorithm.md` of [10]. In L5 and L6 the program completes with owner r (or without owner) if k (or r) does not exist; by Theorem 5 this never happens.

Cost table (Lean bound with $N = n + m + 1$; finer bound written). Stage R: $15N^2$ per round, $O(n^2m)$ in all. Rankings: $14N^2$. Processing order: $15N^3$. Phase 1's picks: $17N^3$. Upgrades: $36N^4, O(n^4 + n^2m)$, the dominant

term. Junk and slots: $O(N^3)$. Owner test: $20N^2 + 24N^3$. Blocks, k^* , chain, rotation, completion: $O(N^3)$. Output: $4N^2$. The total is at most $400N^4$ (`EFX.K3.algoC_cost`), also at most $6400(n+m)^4$. On random instances the count is far below the bound (ratio 0.002 at $n = 30$).

Evidence. Three implementations (a literal transcription of the Lean definitions, the fast Python version, and Lean’s own evaluation of `EFX.K3.algo`) agree on 200,000 random instances (the first two) and on 675 instances (all three), every output passes a raw EFX_0 check, and every Lean operation count is below the proved bound. The fast version takes 1.53s and 1.58s (medians) on random cores with 10^5 agents and 1.5×10^5 and 2×10^5 goods.

D The Lean Formalization

Trusted base. The file `lean/EFX/Model.lean`, copied from [9] with the namespace renamed, defines:

```
def finSum : (k : Nat) → (Fin k → Nat) → Nat
  | 0, _ => 0
  | k+1, f => finSum k (fun i => f i.castSucc)
              + f (Fin.last k)

structure Inst where
  n : Nat
  m : Nat
  v : Fin n → Fin m → Nat
abbrev Alloc := Fin I.m → Fin I.n
def bundleVal (X : I.Alloc) (i j : Fin I.n)
  (ex : Option (Fin I.m)) : Nat :=
  finSum I.m (fun g =>
    if X g = j ∧ ex ≠ some g then I.v i g else 0)
def EFX0 (X : I.Alloc) : Prop :=
  ∀ i j : Fin I.n, i ≠ j → ∀ g : Fin I.m, X g = j →
    I.bundleVal X i j (some g)
    ≤ I.bundleVal X i i none
def numRelevant (I : Inst) (i : Fin I.n) : Nat :=
  finSum I.m (fun g => if 0 < I.v i g then 1 else 0)
```

Main statements.

```
theorem EFX.target (I : Inst) (hn : 0 < I.n)
```

```

(h : ∀ i, numRelevant I i ≤ 3) :
  ∃ X : I.Alloc, I.EFX0 X
theorem EFX.LB.corollaryD (I : Inst) (hn : 0 < I.n)
  (h3 : ∀ i, numRelevant I i = 3)
  (hbal : ∀ i g, 2 * I.v i g ≤ finSum I.m (I.v i)) :
  ∃ X : I.Alloc, I.EFX0 X ∧ ∃ o, ∀ j, j ≠ o →
    finSum I.m (fun g => if X g = j then 1 else 0)
    ≤ 2
def EFX.K3.algo (I : Inst) (hn : 0 < I.n) : I.Alloc :=
  (algoC I hn).val
theorem EFX.K3.algo_efx0 (I : Inst) (hn : 0 < I.n)
  (h : ∀ i, numRelevant I i ≤ 3) :
  I.EFX0 (algo I hn)
theorem EFX.K3.algoC_cost (I : Inst) (hn : 0 < I.n) :
  (algoC I hn).cost ≤ 400 * (I.n + I.m + 1) ^ 4

```

`EFX.target_ordered` and `EFX.corollaryD_ordered` state the same over any type with 0 , $+$ and $\leq$ satisfying the axioms of a linearly ordered cancellative commutative monoid (class `EFX.OrderedValue`), with nonnegativity as a hypothesis, relevance as $v_i(g) > 0$ and balance as $v_i(g) + v_i(g) \leq v_i(M)$; at $\mathbb{N}$ they are exactly the statements above.

Theorems cited (all in `lean/EFX/`; each has a `#print axioms` certificate listing at most the three standard axioms; ledger rows of [10] in brackets):

Result [ledger row]	Lean names (file in <code>lean/EFX/</code>)
Thm. 1 [T]	<code>EFX.target (Target); EFX.target_ordered (RealValues)</code>
Thm. 2 [D]	<code>EFX.LB.corollaryD (CorollaryD); EFX.corollaryD_ordered (RealValues)</code>
Thm. 3 [K3.ALG]	<code>EFX.K3.algo_efx0, EFX.K3.algo_eq_spec (K3Cost)</code>
Thm. [K3.ALG.TIME]	<code>3 EFX.K3.algoC_cost (K3CostBound)</code>
Lem. 1 [L2]	<code>EFX.peel (Peeling); EFX.peelEmpty, EFX.peelR2 (PeelingR2)</code>
Lem. 4 [L3]	<code>EFX.Inst.exists_efx0_of_junk (Junk)</code>
Prop. 2 [T]	<code>EFX.core_reduction (Target)</code>
Lem. 2 [L12]	<code>EFX.l12, EFX.efx0_iff_of_agree (RealValues)</code>
Thm. 4 [S2.LB+]	<code>EFX.LB.Valid.sound (PreAlloc)</code>
Thm. 5 [S2.R]	<code>EFX.LB.theoremA_invalid (OwnerR)</code>
Thm. 6 [S2.LB+]	<code>EFX.LB.theoremB (Rotation)</code>
Thm. 7 [S2.LB+]	<code>EFX.LB.lbPlusRun_sound, EFX.LB.lbPlusOut_exists (LBPlus)</code>
Prop. 1 [K3.OWNER]	<code>EFX.LB.validOwner_iff (OwnerR)</code>
Audit [AUD]	<code>Audit.target_audit, Audit.corollaryD_audit (Audit)</code>
Second [K4.C4X.K3]	<code>proof EFX.C4min.corollaryD_K3, EFX.C4min.target_K3 (K3Theorem)</code>

`EFX.LB.lbPlusRun_sound` quantifies over all runs of LB^+ (any Phase 1 order with R1 priority, any upgrade order, any need chain, any completion satisfying (OC)); `EFX.LB.lbPlusOut_exists` shows that an output always exists. The audit's second kernel (nanoda) checked 4,186 declarations: the 726 named declarations of the library at the time (the natural-number development of Theorems 1 and 2) and their dependencies. The files formalizing Theorems 1–3, their proofs and the audit have about 9,700 lines.

E Computational Certification

Each row is a ledger row of [10] with status CERTIFIED; certificates are stored allocations, re-checked without SAT from the raw EFX_0 definition,

and the enumeration of connected cores is checked for completeness by a second, independent method.

Claim and scope	Size	Row
EFX ₀ exists, every 3-relevant in-stance, $n \leq 6$	251 + 3,182 connected cores	R1
EFX ₀ exists, every 3-relevant in-stance, $n \leq 7$	adds $n = 7$ (rows R2, R5 S2.N7)	
Theorem 2, every core with $n \leq 6$, connected or not	certificates, components combined	R3
Theorem 2, connected cores, $n = 7, 37, 541, 3,103$	cores; 6^7 pro- files each	R2
$11 \leq m \leq 13$		
Theorem 2, connected cores, $n = 8, 1,232$ and 52 cores		R4
$m \in \{14, 15\}$		
LB never fails, cores with $n \leq 6$	3,436 connected, 131 dis-connected	S2.N6
LB never fails, connected cores with $n = 7$	41,170 cores, 1.15×10^{10} pairs	S2.N7
LB never fails, connected cores, $n = 8, m \geq 13$	12,763 cores, 2.14×10^{10} pairs	S2.N8

For the rows on LB, the certificates (LB’s outputs) confirm Theorem 2 in every labelling; that LB itself never fails holds for the labelling stored in the certificates, as LB breaks ties by index. Cores with $m = 2n$ are covered by a two-own-goods argument (every agent holding two of its own goods is safe).

Evidence (not proof). 5,844,263,904 runs of a literal implementation of LB⁺ (every core with $n \leq 6$ under every ranking profile and every sequence of insertion choices; random choices on larger cores; random non-core instances), asserting every lemma of Section 4 and checking every output against the raw definition: 1,327,101 rotations, 0 failures. An independent Python implementation gives identical tallies for $n \leq 5$.

Refutations (status REFUTED, with checkers):

- bundles of at most two goods suffice in a core: false for $n = 3, m = 5$ (Proposition 3(a), 14 of 216 profiles); no core with $n = 2$ fails;
- bundles of at most two goods suffice when $m \leq 2n - 2$: false, smallest $n = 4, m = 6$; at $n = 6, m = 10$, 57 of the 211 connected cores have a failing profile (confirmed by a second encoding);

- the same for instances that are not cores: four identical agents with values 4, 3, 2 on three goods, and three goods valued by nobody;
- one bundle of three goods suffices: false for $n = 4$, $m = 7$ (Proposition 3(b)); at $n = 6$, $m = 11$, 3 of 25 cores need four.

F Ordinality of Cores and the Counterexamples

A good is *alone* if it is the only good of its bundle.

Lemma 6 (cases of safety). *Let agent i have $R_i = \{a, b, c\}$ with $v_i(a) > v_i(b) > v_i(c) > 0$ and $v_i(a) < v_i(b) + v_i(c)$. Then i satisfies the EFX_0 constraints iff one of: (T) i holds a , and either i holds b or c , or b and c are not together in a bundle of at least three goods; (P) i holds b and c ; (B) i holds b , and a is alone; (C) i holds c , and a and b are alone; (E) a , b and c are all alone.*

Proof. The subset sums are ordered $0 < c < b < a < b + c < a + c < a + b < a + b + c$ (writing x for $v_i(x)$), so every constraint is decided by the ranking. Consider the goods of R_i that i holds. If it holds a and one more, every other bundle meets R_i in one good, worth $\leq b < a + c$. If it holds $\{b, c\}$, the only threat is $a < b + c$. If it holds only a : bundles meeting R_i in one good are harmless; $\{b, c\}$ as a bundle threatens $b < a$; b, c together with another good threaten $b + c > a$. If it holds only b : a not alone lies with c (threat a) or with a good outside R_i (threat $\geq a$), both $> b$; if a is alone, c 's bundle threatens at most $c < b$. If it holds only c , it needs a and b alone by the same argument. If it holds none of a, b, c , any bundle with a good of R_i and at least two goods threatens a positive amount, so all three must be alone. $\square$

The cases remain sufficient with ties, so tied cores reduce to strict ranking profiles (`proofs/lemmas.md`, L5).

Proposition 3 (limits of the shape). *Take strict rankings and strictly balanced values. (a) Let agents 0, 1, 2 rank $g_0 \succ g_1 \succ p_i$, where p_i is private to agent i . No EFX_0 allocation has all bundles of at most two goods, while $\{g_0\}, \{g_1\}, \{p_0, p_1, p_2\}$ is EFX_0 . (b) Let agents 0, 1, 2, 3 rank $g_0 \succ g_2 \succ p_0$, $g_0 \succ g_2 \succ p_1$, $g_1 \succ g_2 \succ p_2$, $g_1 \succ g_2 \succ p_3$. Every EFX_0 allocation with at most one bundle of more than two goods has bundle sizes 4, 1, 1, 1.*

Exhaustive computation shows that no core with two agents needs a bundle of more than two goods, so (a) is a smallest example (ledger row X1 of [10]; `proofs/counterexamples.md` and `proofs/construction.md`, Section 1).

Proof (of Proposition 3). (a) With five goods in three bundles of at most two goods, the sizes are 2, 2, 1, so exactly one good is alone. Good g_0 is everyone's top and has one holder, so two agents do not hold their top. For them (T) fails, (C) and (E) need two or three alone goods, so each is in case (P) or (B), and both hold g_1 : impossible. In $\{g_0\}, \{g_1\}, \{p_0, p_1, p_2\}$, the holder of g_0 is in case (T), the holder of g_1 in case (B), and the third agent in case (C). (b) Each of the tops g_0, g_1 has a claimant that does not hold it (a *loser*). A loser is safe only by (P) or (B), which need it to hold g_2 , or by (C) or (E), which need g_2 alone. The two losers are distinct and cannot both hold g_2 , so g_2 is alone, and no agent is in case (P). A loser holding $\{g_2\}$ is in case (B) or (E) and needs its top alone; a loser not holding g_2 is in case (C) or (E) and needs its top and g_2 alone. So g_0, g_1, g_2 are alone, held by three agents, and the fourth bundle is $\{p_0, p_1, p_2, p_3\}$. The allocation giving g_0 to agent 0, g_1 to agent 2, g_2 to agent 1 and the private goods to agent 3 is EFX₀ (cases T, B, T, C). $\square$